PYTHON ACTIVITY-BASED LEARNING REPORT

Name: Charan Teja	**Roll No:** 26000191
Branch: CSE	**Course:** Python Programming

Case 1: Scholarship Eligibility and Ranking

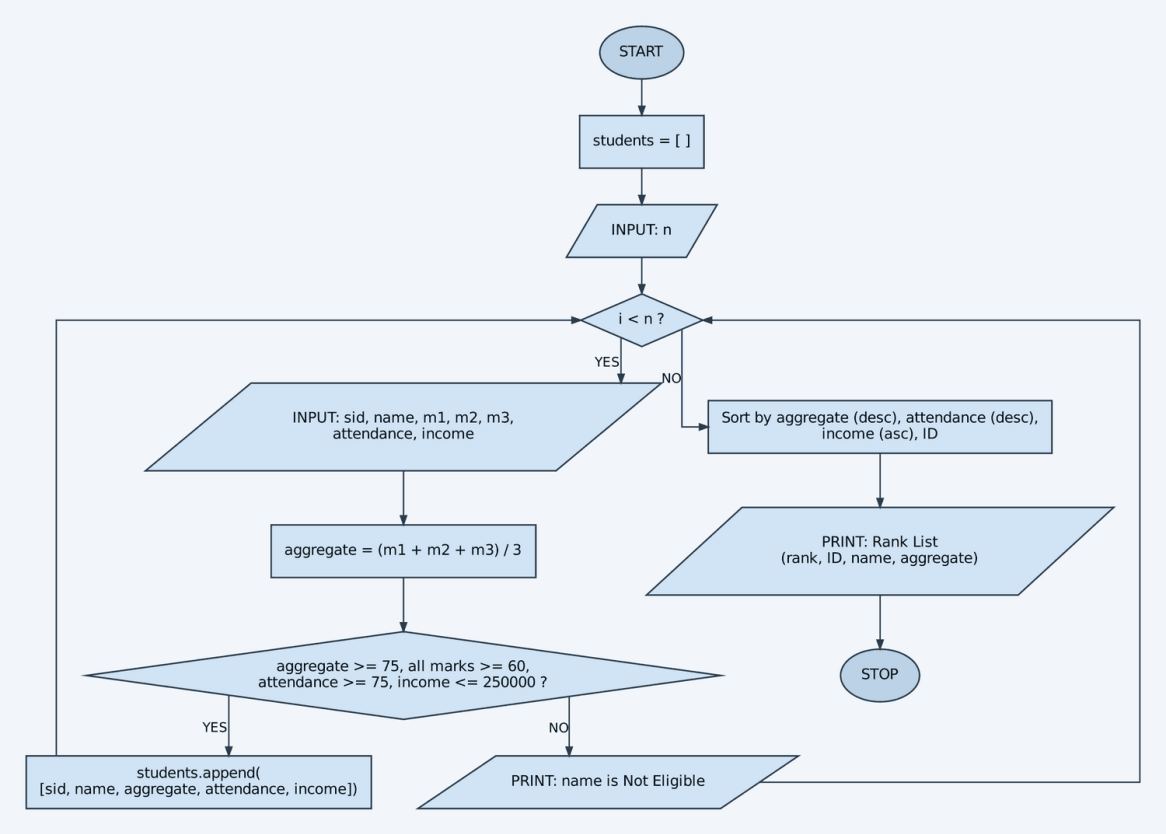
Problem Statement

Write a Python program that reads the details of n students (ID, name, marks in Python, Maths and English, attendance and annual family income) and checks scholarship eligibility. A student is eligible only if the aggregate is at least 75, every subject mark is at least 60, attendance is at least 75% and annual family income is at most Rs. 250000. Eligible students are ranked by aggregate (highest first), then attendance (highest first), then income (lowest first) and finally by student ID.

Algorithm

- Step 1: Start.
- Step 2: Create an empty list students and read the number of students n.
- Step 3: For each student, read ID, name, three subject marks, attendance and annual family income.
- Step 4: Calculate aggregate = (m1 + m2 + m3) / 3.
- Step 5: If aggregate >= 75, all marks >= 60, attendance >= 75 and income <= 250000, append the student to the list; otherwise print that the student is not eligible.
- Step 6: Sort the list by aggregate (descending), attendance (descending), income (ascending) and ID.
- Step 7: Print the rank list with rank, ID, name and aggregate. Stop.

Flowchart



Case 1: Scholarship Eligibility and Ranking (continued)

Source Code & Output

```
students = []
n = int(input("Enter number of students: "))
for i in range(n):
    print("\nStudent", i + 1)
    sid = input("Student ID: ")
    name = input("Name: ")
    m1 = float(input("Python marks: "))
    m2 = float(input("Maths marks: "))
    m3 = float(input("English marks: "))

    attendance = float(input("Attendance: "))
    income = float(input("Annual family income: "))

    aggregate = (m1 + m2 + m3) / 3

    if (aggregate >= 75 and m1 >= 60 and m2 >= 60
            and m3 >= 60 and attendance >= 75
            and income <= 250000):

        students.append(
            [sid, name, aggregate, attendance, income]
        )
    else:
        print(name, "is Not Eligible")
students.sort(key=lambda s: (-s[2], -s[3], s[4], s[0]))

print("\nScholarship Rank List")
print("Rank  ID     Name     Aggregate")

for rank, s in enumerate(students, 1):
    print(rank, s[0], s[1], round(s[2], 2))
```

Output

```
Enter number of students: 3

Student 1
Student ID: S101
Name: Asha
Python marks: 85
Maths marks: 80
English marks: 90
Attendance: 90
Annual family income: 200000

Student 2
Student ID: S102
Name: Ravi
Python marks: 70
Maths marks: 65
English marks: 72
Attendance: 80
Annual family income: 150000
Ravi is Not Eligible

Student 3
Student ID: S103
Name: Kiran
Python marks: 90
Maths marks: 85
English marks: 88
Attendance: 95
Annual family income: 180000

Scholarship Rank List
Rank  ID     Name     Aggregate
1 S103 Kiran 87.67
2 S101 Asha 85.0
```

Case 2: Attendance Recovery Planner

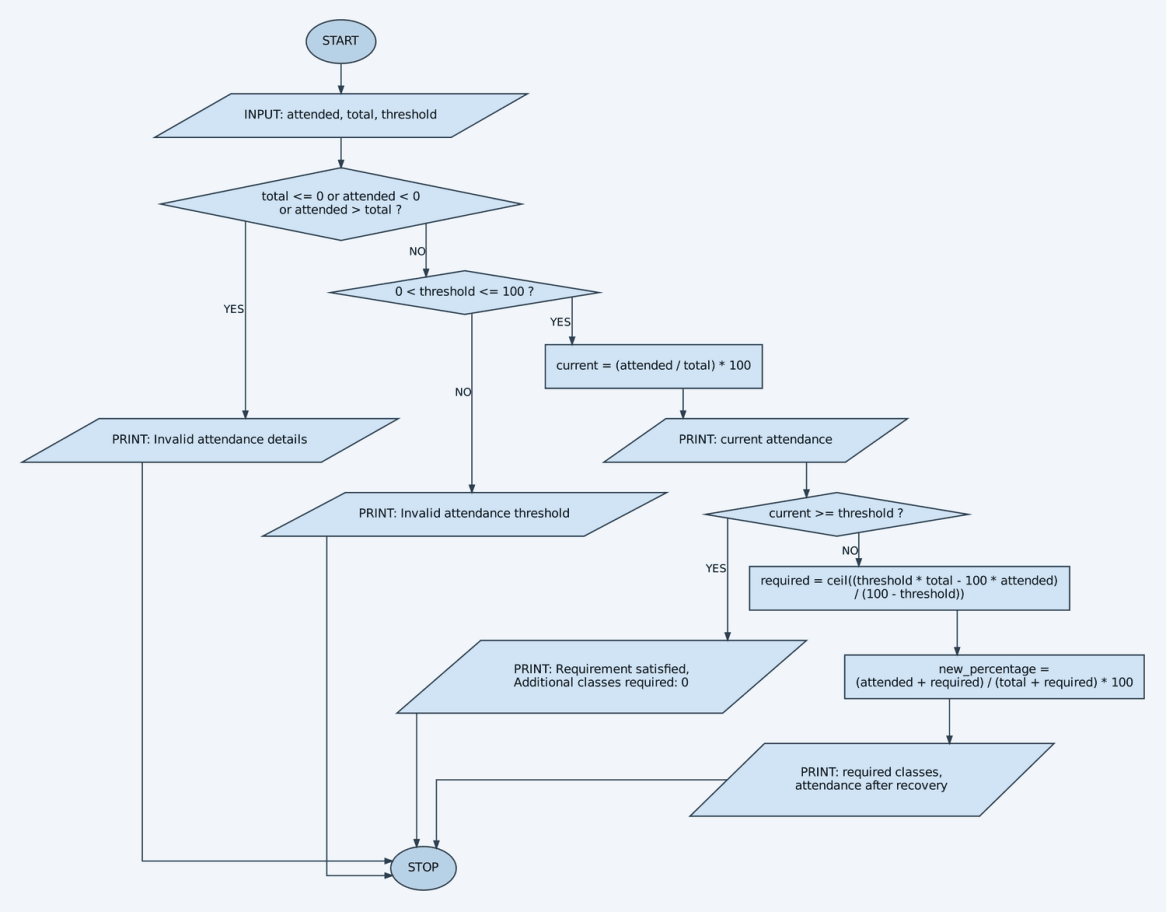
Problem Statement

Calculate the minimum number of consecutive classes a student must attend to reach a required attendance percentage, given the classes attended, the total classes held and the required percentage. The inputs must be validated, and the program must report the current attendance and the attendance after recovery.

Algorithm

- Step 1: Start.
- Step 2: Read classes attended, total classes and the required attendance percentage.
- Step 3: If total <= 0, attended < 0 or attended > total, print "Invalid attendance details" and stop.
- Step 4: If the threshold is not in the range 0 < threshold <= 100, print "Invalid attendance threshold" and stop.
- Step 5: Calculate current = (attended / total) x 100 and print it.
- Step 6: If current >= threshold, print that the requirement is already satisfied (0 additional classes).
- Step 7: Else required = ceil((threshold x total - 100 x attended) / (100 - threshold)).
- Step 8: Calculate new_percentage = (attended + required) / (total + required) x 100.
- Step 9: Print the required classes and the attendance after recovery. Stop.

Flowchart



Case 2: Attendance Recovery Planner (continued)

Source Code & Output

```
import math
attended = int(input("Enter classes attended: "))
total = int(input("Enter total classes: "))
threshold = float(input("Enter required attendance percentage: "))
if total <= 0 or attended < 0 or attended > total:
    print("Invalid attendance details")
elif not 0 < threshold <= 100:
    print("Invalid attendance threshold")
else:
    current = (attended / total) * 100
    print("Current attendance:", round(current, 2), "%")
    if current >= threshold:
        print("Attendance requirement already satisfied")
        print("Additional classes required: 0")
    else:
        required = math.ceil(
            (threshold * total - 100 * attended)
            / (100 - threshold)
        )

        print("Minimum consecutive classes to attend:", required)

        new_percentage = (
            (attended + required) / (total + required)
        ) * 100

        print("Attendance after recovery:", round(new_percentage, 2), "%")
```

Output

```
Enter classes attended: 30
Enter total classes: 50
Enter required attendance percentage: 75
Current attendance: 60.0 %
Minimum consecutive classes to attend: 30
Attendance after recovery: 75.0 %
```

Case 3: Chemical Inventory Validation

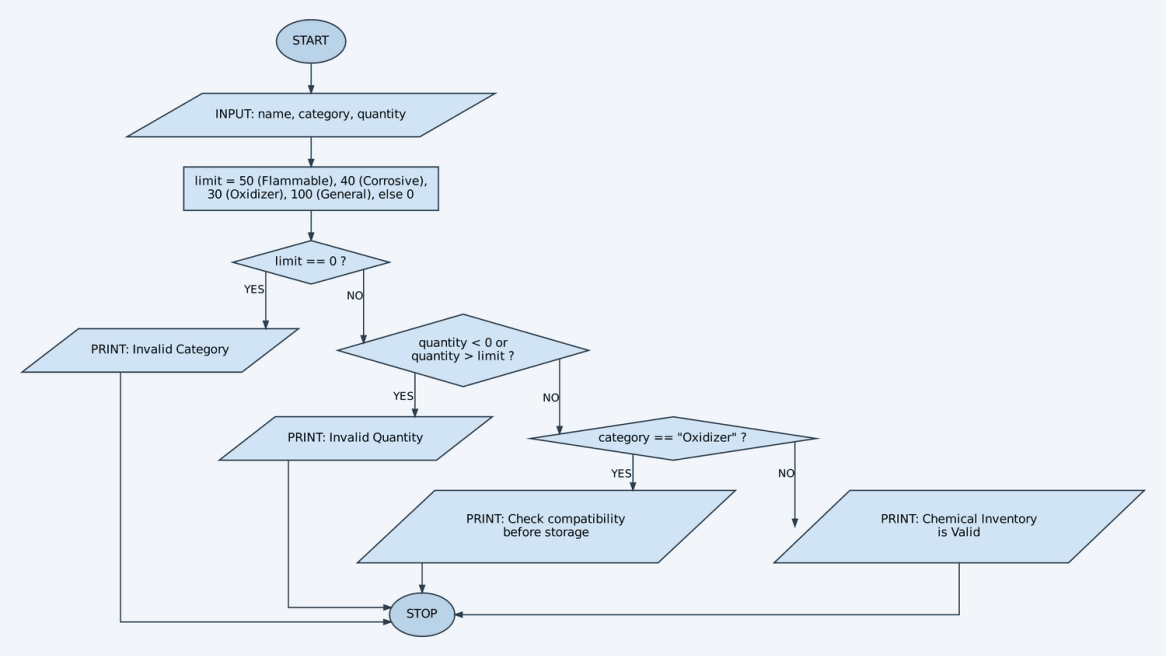
Problem Statement

Validate the storage of a chemical in a laboratory inventory based on its category and quantity. The storage limits are: Flammable - 50, Corrosive - 40, Oxidizer - 30 and General - 100. Any other category is invalid, a quantity that is negative or above the limit is invalid, and oxidizers need a compatibility check before storage.

Algorithm

- Step 1: Start.
- Step 2: Read the chemical name, category and quantity.
- Step 3: Set limit = 50 for Flammable, 40 for Corrosive, 30 for Oxidizer, 100 for General; otherwise limit = 0.
- Step 4: If limit = 0, print "Invalid Category".
- Step 5: Else if quantity < 0 or quantity > limit, print "Invalid Quantity".
- Step 6: Else if the category is Oxidizer, print "Check compatibility before storage".
- Step 7: Otherwise print "Chemical Inventory is Valid". Stop.

Flowchart



Case 3: Chemical Inventory Validation (continued)

Source Code & Output

```
name = input("Enter chemical name: ")
category = input("Enter category: ")
quantity = float(input("Enter quantity: "))

if category == "Flammable":
    limit = 50
elif category == "Corrosive":
    limit = 40
elif category == "Oxidizer":
    limit = 30
elif category == "General":
    limit = 100
else:
    limit = 0

if limit == 0:
    print("Invalid Category")
elif quantity < 0 or quantity > limit:
    print("Invalid Quantity")
elif category == "Oxidizer":
    print("Check compatibility before storage")
else:
    print("Chemical Inventory is Valid")
```

Output

```
Enter chemical name: Sulphuric Acid
Enter category: Corrosive
Enter quantity: 35
Chemical Inventory is Valid
```

Case 4: Campus Registration ID Analyzer

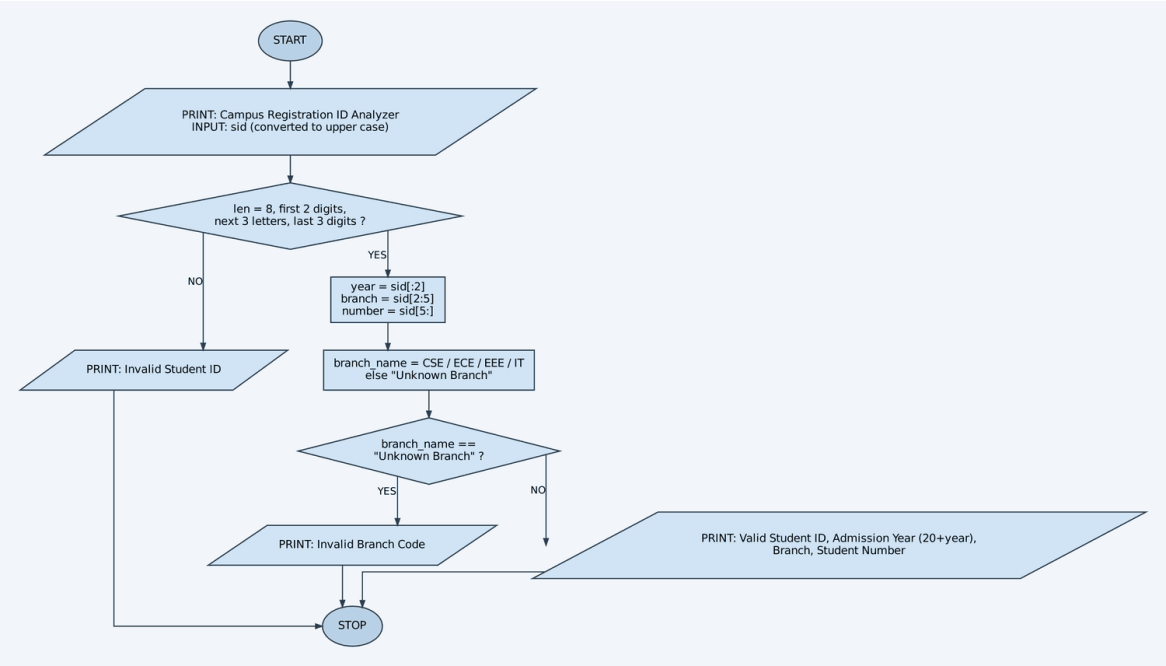
Problem Statement

Analyze a campus student registration ID of the form YYBBBNNN: the first two digits give the admission year, the next three letters give the branch code (CSE, ECE, EEE, IT) and the last three digits give the student number. The program must validate the format, identify the branch and report the details, or report an invalid ID / invalid branch code.

Algorithm

- Step 1: Start and print the title.
- Step 2: Read the student ID and convert it to upper case.
- Step 3: Check that the length is 8, the first 2 characters are digits, the next 3 are letters and the last 3 are digits.
- Step 4: If the format is wrong, print "Invalid Student ID" and stop.
- Step 5: Extract year = sid[:2], branch = sid[2:5] and number = sid[5:].
- Step 6: Map the branch code to its name (CSE, ECE, EEE, IT); otherwise branch name = "Unknown Branch".
- Step 7: If the branch is unknown, print "Invalid Branch Code".
- Step 8: Otherwise print the admission year (20 + year), the branch name and the student number. Stop.

Flowchart



Case 4: Campus Registration ID Analyzer (continued)

Source Code & Output

```
print("Campus Registration ID Analyzer")

sid = input("Enter Student ID: ").upper()

if (len(sid) == 8 and sid[:2].isdigit()
        and sid[2:5].isalpha() and sid[5:].isdigit()):

    year = sid[:2]
    branch = sid[2:5]
    number = sid[5:]

    if branch == "CSE":
        branch_name = "Computer Science"
    elif branch == "ECE":
        branch_name = "Electronics"
    elif branch == "EEE":
        branch_name = "Electrical"
    elif branch == "IT":
        branch_name = "Information Technology"
    else:
        branch_name = "Unknown Branch"

    if branch_name == "Unknown Branch":
        print("Invalid Branch Code")
    else:
        print("Valid Student ID")
        print("Admission Year: 20" + year)
        print("Branch:", branch_name)
        print("Student Number:", number)

else:
    print("Invalid Student ID")
```

Output

```
Campus Registration ID Analyzer
Enter Student ID: 23cse045
Valid Student ID
Admission Year: 2023
Branch: Computer Science
Student Number: 045
```

Case 5: Student Marks Analysis using 2D Lists

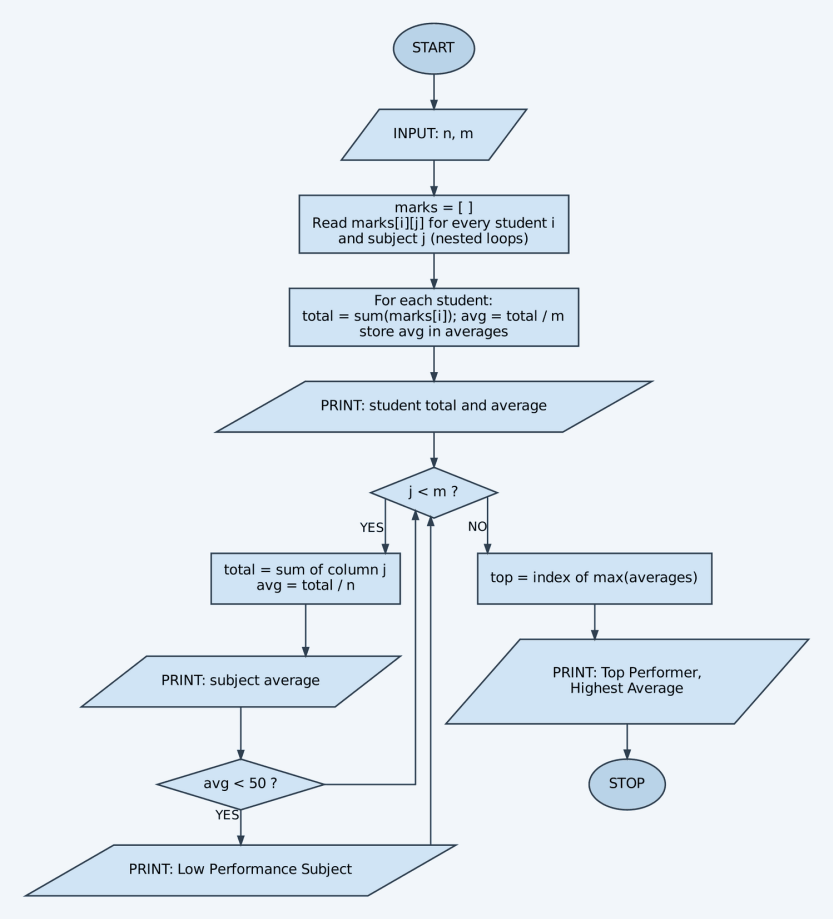
Problem Statement

Read the marks of n students in m subjects into a two-dimensional list. Calculate the total and average of every student and the average of every subject, flag a subject whose average is below 50 as a Low Performance Subject, and find the top performer (the student with the highest average).

Algorithm

- Step 1: Start.
- Step 2: Read the number of students n and the number of subjects m.
- Step 3: Using nested loops, read the marks of every student and store them in the 2D list marks.
- Step 4: For each student, calculate total = sum(marks[i]) and avg = total / m; store the average and print both.
- Step 5: For each subject j, add the marks of all students and calculate avg = total / n; print it.
- Step 6: If the subject average is below 50, print "Low Performance Subject".
- Step 7: Find the index of the maximum student average and print the top performer and the highest average. Stop.

Flowchart



Case 5: Student Marks Analysis using 2D Lists (continued)

Source Code & Output

```
n = int(input("Enter number of students: "))
m = int(input("Enter number of subjects: "))
marks = []
for i in range(n):
    row = []
    print("Enter marks of Student", i + 1)
    for j in range(m):
        mark = int(input("Enter subject mark: "))
        row.append(mark)
    marks.append(row)
print("\nStudent Total and Average")
averages = []
for i in range(n):
    total = sum(marks[i])
    avg = total / m
    averages.append(avg)
    print("Student", i + 1, "Total:", total,
          "Average:", round(avg, 2))
print("\nSubject Averages")
for j in range(m):
    total = 0
    for i in range(n):
        total += marks[i][j]
    avg = total / n
    print("Subject", j + 1, "Average:", round(avg, 2))
    if avg < 50:
        print("Low Performance Subject")
top = averages.index(max(averages))
print("\nTop Performer: Student", top + 1)
print("Highest Average:", round(averages[top], 2))
```

Output

```
Enter number of students: 3
Enter number of subjects: 3
Enter marks of Student 1
Enter subject mark: 60
Enter subject mark: 70
Enter subject mark: 40
Enter marks of Student 2
Enter subject mark: 35
Enter subject mark: 45
Enter subject mark: 30
Enter marks of Student 3
Enter subject mark: 80
Enter subject mark: 85
Enter subject mark: 45

Student Total and Average
Student 1 Total: 170 Average: 56.67
Student 2 Total: 110 Average: 36.67
Student 3 Total: 210 Average: 70.0

Subject Averages
Subject 1 Average: 58.33
Subject 2 Average: 66.67
Subject 3 Average: 38.33
Low Performance Subject

Top Performer: Student 3
Highest Average: 70.0
```

Case 6: Food Waste Analysis

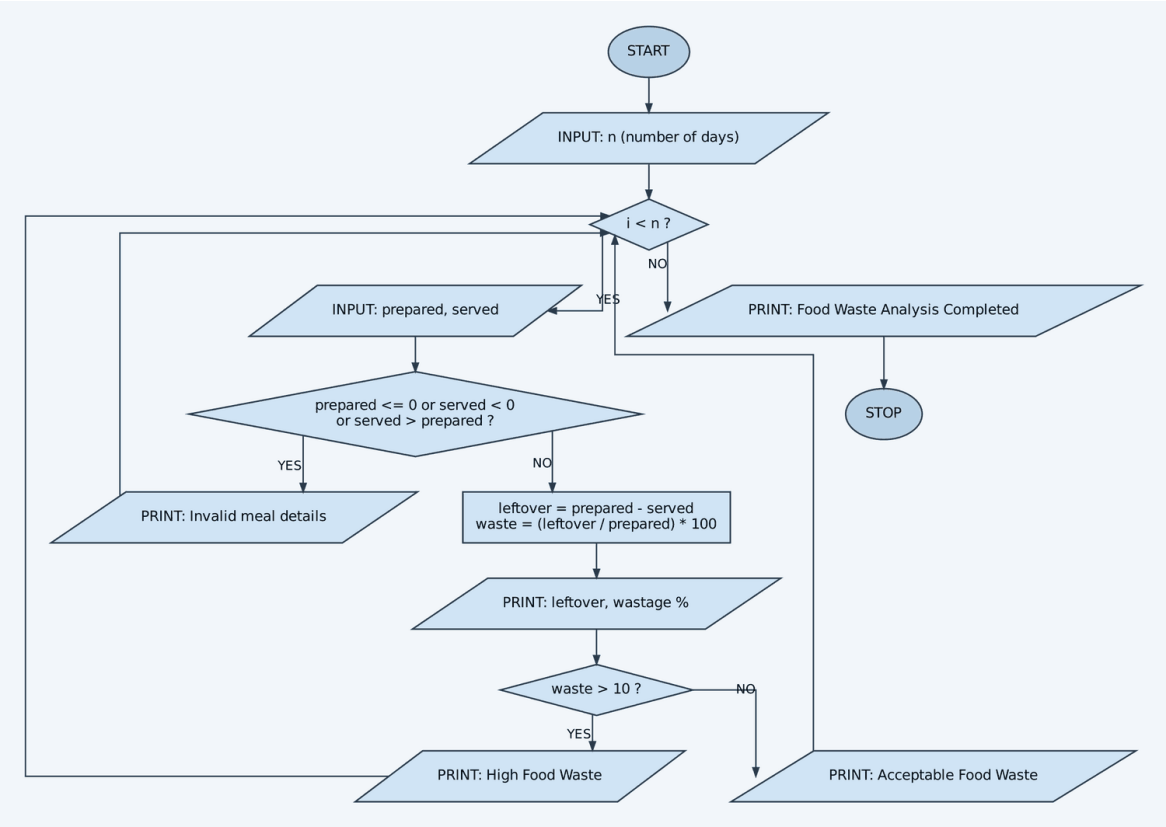
Problem Statement

A hostel mess records the number of meals prepared and served every day. For n days, calculate the leftover meals and the wastage percentage, and report "High Food Waste" when the wastage is more than 10% and "Acceptable Food Waste" otherwise. Invalid entries (prepared <= 0, served < 0 or served > prepared) must be reported and skipped.

Algorithm

- Step 1: Start.
- Step 2: Read the number of days n.
- Step 3: For each day, read the prepared meals and the served meals.
- Step 4: If prepared <= 0, served < 0 or served > prepared, print "Invalid meal details" and continue with the next day.
- Step 5: Calculate leftover = prepared - served and waste = (leftover / prepared) x 100.
- Step 6: Print the leftover meals and the wastage percentage.
- Step 7: If waste > 10, print "High Food Waste"; otherwise print "Acceptable Food Waste".
- Step 8: After all days, print "Food Waste Analysis Completed". Stop.

Flowchart



Case 6: Food Waste Analysis (continued)

Source Code & Output

```
n = int(input("Enter number of days: "))

for i in range(n):
    print("\nDay", i + 1)

    prepared = int(input("Enter prepared meals: "))
    served = int(input("Enter served meals: "))

    if prepared <= 0 or served < 0 or served > prepared:
        print("Invalid meal details")
        continue

    leftover = prepared - served
    waste = (leftover / prepared) * 100

    print("Leftover meals:", leftover)
    print("Wastage percentage:", round(waste, 2), "%")

    if waste > 10:
        print("High Food Waste")
    else:
        print("Acceptable Food Waste")

print("\nFood Waste Analysis Completed")
```

Output

```
Enter number of days: 3

Day 1
Enter prepared meals: 100
Enter served meals: 95
Leftover meals: 5
Wastage percentage: 5.0 %
Acceptable Food Waste

Day 2
Enter prepared meals: 120
Enter served meals: 100
Leftover meals: 20
Wastage percentage: 16.67 %
High Food Waste

Day 3
Enter prepared meals: 80
Enter served meals: 80
Leftover meals: 0
Wastage percentage: 0.0 %
Acceptable Food Waste

Food Waste Analysis Completed
```

Case 7: Computer Resource Allocation

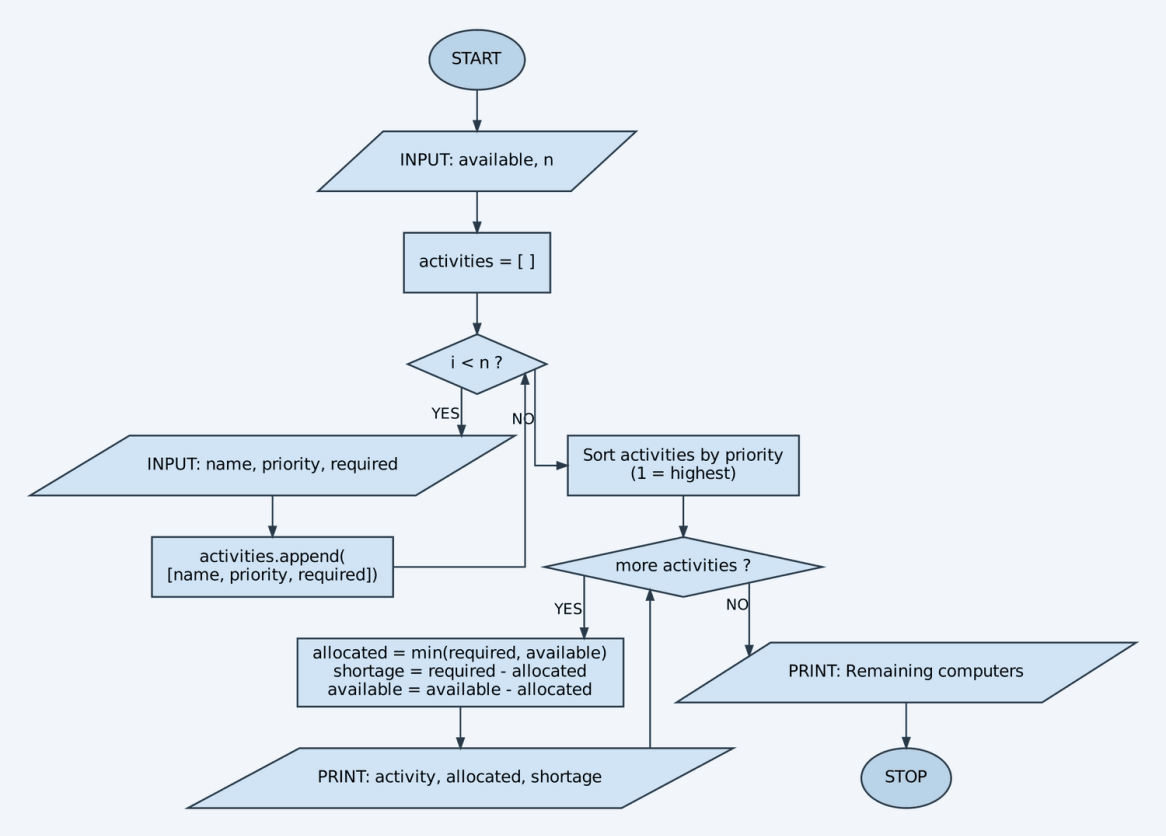
Problem Statement

A computer lab has a limited number of computers that must be shared among several activities. Each activity has a priority (1 is the highest) and a number of required computers. Allocate computers in priority order, giving each activity the smaller of its requirement and the computers still available, and report the allocation, the shortage of every activity and the computers remaining.

Algorithm

- Step 1: Start.
- Step 2: Read the number of available computers and the number of activities n.
- Step 3: For each activity read its name, priority and required computers and store them in a list.
- Step 4: Sort the activities by priority (1 = highest first).
- Step 5: For each activity, allocated = min(required, available) and shortage = required - allocated.
- Step 6: Reduce available by the allocated computers and print the activity, allocated and shortage.
- Step 7: After all activities, print the remaining computers. Stop.

Flowchart



Case 7: Computer Resource Allocation (continued)

Source Code & Output

```
available = int(input("Enter available computers: "))
n = int(input("Enter number of activities: "))
activities = []
for i in range(n):
    print("\nActivity", i + 1)

    name = input("Enter activity name: ")
    priority = int(input("Enter priority (1 is highest): "))
    required = int(input("Enter required computers: "))

    activities.append([name, priority, required])
activities.sort(key=lambda x: x[1])

print("\nResource Allocation Results")

for a in activities:
    name = a[0]
    required = a[2]

    allocated = min(required, available)
    shortage = required - allocated

    available = available - allocated

    print("\nActivity:", name)
    print("Allocated:", allocated)
    print("Shortage:", shortage)

print("\nRemaining computers:", available)
```

Output

```
Enter available computers: 20
Enter number of activities: 3

Activity 1
Enter activity name: Lab Exam
Enter priority (1 is highest): 1
Enter required computers: 10

Activity 2
Enter activity name: Project Demo
Enter priority (1 is highest): 2
Enter required computers: 8

Activity 3
Enter activity name: Coding Workshop
Enter priority (1 is highest): 3
Enter required computers: 6

Resource Allocation Results

Activity: Lab Exam
Allocated: 10
Shortage: 0

Activity: Project Demo
Allocated: 8
Shortage: 0

Activity: Coding Workshop
Allocated: 2
Shortage: 4

Remaining computers: 0
```

Case 8: Festival Financial Report

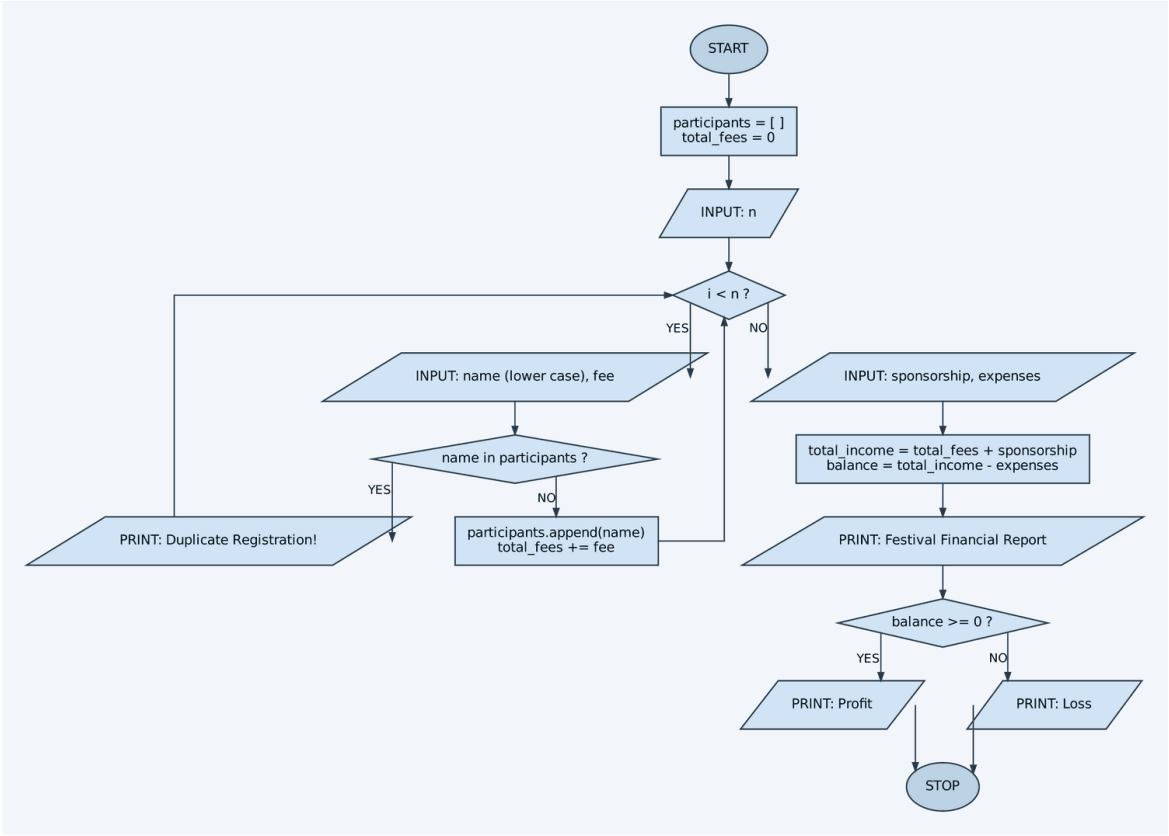
Problem Statement

Prepare the financial report of a college festival. Read the names and registration fees of the participants (a repeated name, ignoring case, is a duplicate registration and is not counted), then read the sponsorship amount and the total expenses. Calculate the total income and the final balance, and report whether the festival ended in profit or loss.

Algorithm

- Step 1: Start.
- Step 2: Create an empty list participants, set total_fees = 0 and read the number of participants n.
- Step 3: For each participant read the name (converted to lower case) and the registration fee.
- Step 4: If the name is already in the list, print "Duplicate Registration!"; otherwise add the name to the list and add the fee to total_fees.
- Step 5: Read the sponsorship amount and the total expenses.
- Step 6: Calculate total_income = total_fees + sponsorship and balance = total_income - expenses.
- Step 7: Print the total participants, fees, sponsorship, income, expenses and the final balance.
- Step 8: If balance >= 0, print "Profit"; otherwise print "Loss". Stop.

Flowchart



Case 8: Festival Financial Report (continued)

Source Code & Output

```
participants = []
total_fees = 0
n = int(input("Enter number of participants: "))

for i in range(n):
    name = input("Enter participant name: ").lower()
    fee = float(input("Enter registration fee: "))

    if name in participants:
        print("Duplicate Registration!")
    else:
        participants.append(name)
        total_fees += fee

sponsorship = float(input("Enter sponsorship amount: "))
expenses = float(input("Enter total expenses: "))

total_income = total_fees + sponsorship
balance = total_income - expenses

print("\nFestival Financial Report")

print("Total Participants:", len(participants))
print("Total Registration Fees:", total_fees)
print("Sponsorship Amount:", sponsorship)
print("Total Income:", total_income)
print("Total Expenses:", expenses)
print("Final Balance:", balance)

if balance >= 0:
    print("Financial Status: Profit")
else:
    print("Financial Status: Loss")
```

Output

```
Enter number of participants: 4
Enter participant name: Anil
Enter registration fee: 200
Enter participant name: Sita
Enter registration fee: 200
Enter participant name: ANIL
Enter registration fee: 200
Duplicate Registration!
Enter participant name: Ravi
Enter registration fee: 250
Enter sponsorship amount: 5000
Enter total expenses: 4000

Festival Financial Report
Total Participants: 3
Total Registration Fees: 650.0
Sponsorship Amount: 5000.0
Total Income: 5650.0
Total Expenses: 4000.0
Final Balance: 1650.0
Financial Status: Profit
```

Case 9: Festival Financial Report (Loss Scenario)

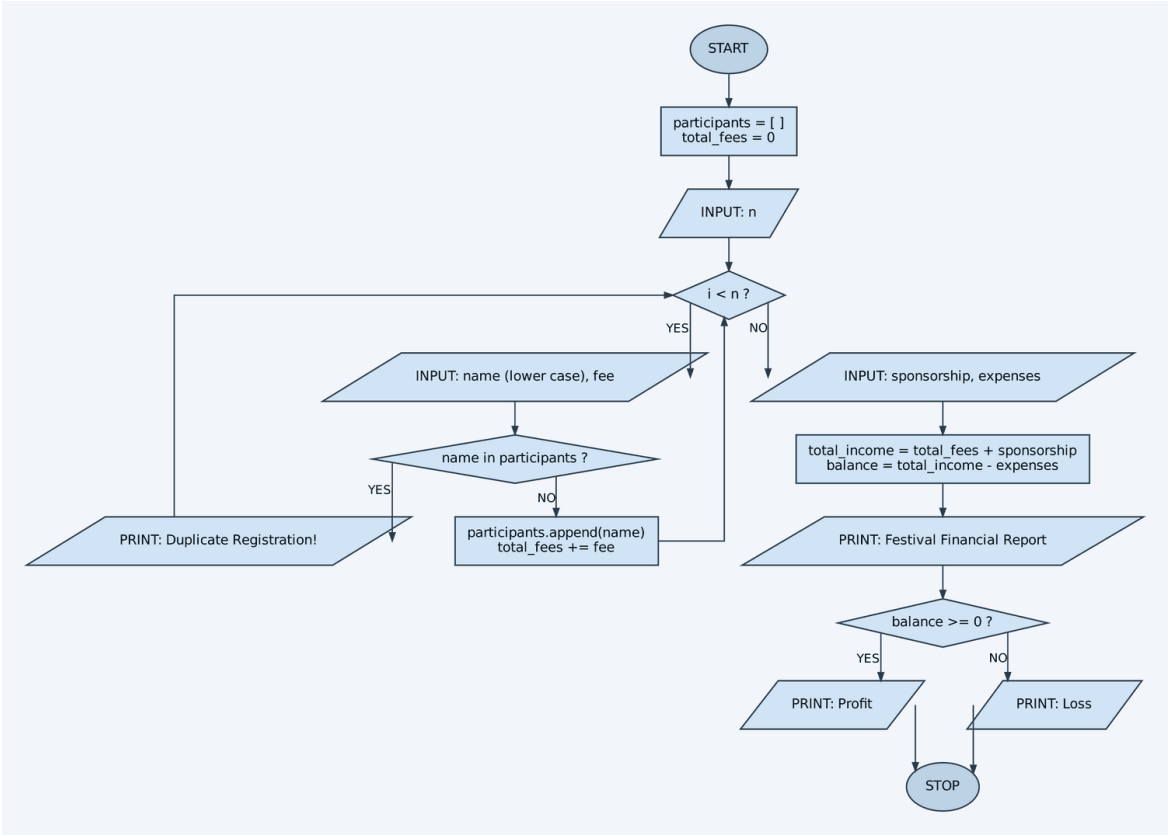
Problem Statement

Prepare the financial report of a college festival. Read the names and registration fees of the participants (a repeated name, ignoring case, is a duplicate registration and is not counted), then read the sponsorship amount and the total expenses. Calculate the total income and the final balance, and report whether the festival ended in profit or loss.

Algorithm

- Step 1: Start.
- Step 2: Create an empty list participants, set total_fees = 0 and read the number of participants n.
- Step 3: For each participant read the name (converted to lower case) and the registration fee.
- Step 4: If the name is already in the list, print "Duplicate Registration!"; otherwise add the name to the list and add the fee to total_fees.
- Step 5: Read the sponsorship amount and the total expenses.
- Step 6: Calculate total_income = total_fees + sponsorship and balance = total_income - expenses.
- Step 7: Print the total participants, fees, sponsorship, income, expenses and the final balance.
- Step 8: If balance >= 0, print "Profit"; otherwise print "Loss". Stop.

Flowchart



Case 9: Festival Financial Report (Loss Scenario) (continued)

Source Code & Output

```
participants = []
total_fees = 0
n = int(input("Enter number of participants: "))

for i in range(n):
    name = input("Enter participant name: ").lower()
    fee = float(input("Enter registration fee: "))

    if name in participants:
        print("Duplicate Registration!")
    else:
        participants.append(name)
        total_fees += fee

sponsorship = float(input("Enter sponsorship amount: "))
expenses = float(input("Enter total expenses: "))

total_income = total_fees + sponsorship
balance = total_income - expenses

print("\nFestival Financial Report")

print("Total Participants:", len(participants))
print("Total Registration Fees:", total_fees)
print("Sponsorship Amount:", sponsorship)
print("Total Income:", total_income)
print("Total Expenses:", expenses)
print("Final Balance:", balance)

if balance >= 0:
    print("Financial Status: Profit")
else:
    print("Financial Status: Loss")
```

Output

```
Enter number of participants: 4
Enter participant name: Meena
Enter registration fee: 150
Enter participant name: Kiran
Enter registration fee: 150
Enter participant name: meena
Enter registration fee: 150
Duplicate Registration!
Enter participant name: Suresh
Enter registration fee: 200
Enter sponsorship amount: 1000
Enter total expenses: 2500

Festival Financial Report
Total Participants: 3
Total Registration Fees: 500.0
Sponsorship Amount: 1000.0
Total Income: 1500.0
Total Expenses: 2500.0
Final Balance: -1000.0
Financial Status: Loss
```

Case 10: ATM Transaction Audit

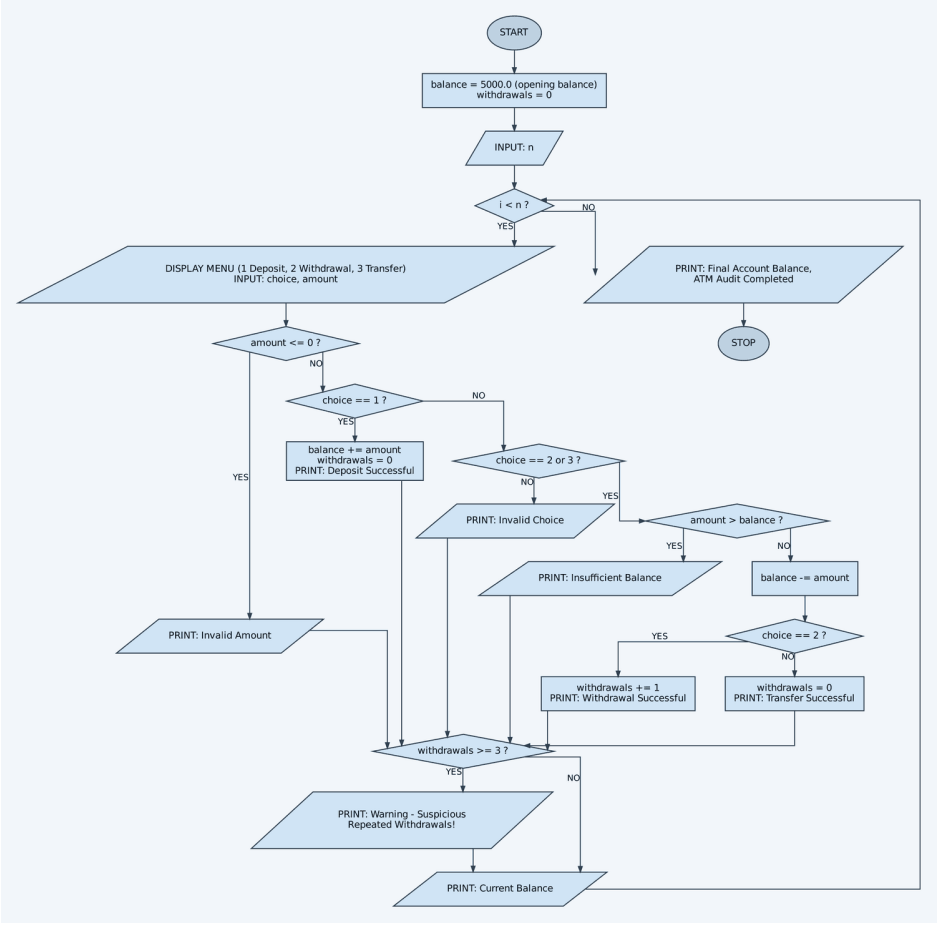
Problem Statement

Simulate and audit n ATM transactions (Deposit, Withdrawal and Transfer) on an account. Validate every amount, reject withdrawals and transfers that exceed the balance, reject invalid menu choices, and print a warning whenever three or more withdrawals happen consecutively. At the end print the final account balance.

Algorithm

- Step 1: Start.
- Step 2: Set the opening balance, set withdrawals = 0 and read the number of transactions n.
- Step 3: For each transaction, display the menu (1. Deposit, 2. Withdrawal, 3. Transfer) and read the choice and the amount.
- Step 4: If amount <= 0, print "Invalid Amount".
- Step 5: Else if choice = 1, add the amount to the balance, reset withdrawals to 0 and print "Deposit Successful".
- Step 6: Else if choice is 2 or 3: if amount > balance print "Insufficient Balance"; otherwise subtract the amount. For a withdrawal, increase withdrawals by 1; for a transfer, reset withdrawals to 0.
- Step 7: For any other choice print "Invalid Choice".
- Step 8: If withdrawals >= 3, print the suspicious repeated withdrawals warning. Print the current balance.
- Step 9: After all transactions, print the final balance and "ATM Audit Completed". Stop.

Flowchart



Case 10: ATM Transaction Audit (continued)

Source Code & Output

```
balance = 5000.0
n = int(input("Enter number of transactions: "))
withdrawals = 0
for i in range(n):
    print("\n1. Deposit")
    print("2. Withdrawal")
    print("3. Transfer")
    choice = int(input("Enter your choice: "))
    amount = float(input("Enter amount: "))
    if amount <= 0:
        print("Invalid Amount")
    elif choice == 1:
        balance += amount
        withdrawals = 0
        print("Deposit Successful")
    elif choice == 2 or choice == 3:
        if amount > balance:
            print("Insufficient Balance")
        else:
            balance -= amount
            if choice == 2:
                withdrawals += 1
                print("Withdrawal Successful")
            else:
                withdrawals = 0
                print("Transfer Successful")
    else:
        print("Invalid Choice")

    if withdrawals >= 3:
        print("Warning: Suspicious Repeated Withdrawals!")

    print("Current Balance:", balance)

print("\nFinal Account Balance:", balance)
print("ATM Audit Completed")
```

Output

```
Enter number of transactions: 5

1. Deposit
2. Withdrawal
3. Transfer
Enter your choice: 1
Enter amount: 2000
Deposit Successful
Current Balance: 7000.0

1. Deposit
2. Withdrawal
3. Transfer
Enter your choice: 2
Enter amount: 500
Withdrawal Successful
Current Balance: 6500.0

1. Deposit
2. Withdrawal
3. Transfer
Enter your choice: 2
Enter amount: 500
Withdrawal Successful
Current Balance: 6000.0

1. Deposit
2. Withdrawal
3. Transfer
Enter your choice: 2
Enter amount: 500
Withdrawal Successful
Warning: Suspicious Repeated Withdrawals!
Current Balance: 5500.0

1. Deposit
2. Withdrawal
3. Transfer
Enter your choice: 3
Enter amount: 1000
Transfer Successful
Current Balance: 4500.0

Final Account Balance: 4500.0
ATM Audit Completed
```